

Aufgabe 1 *Sequentielle Logik*

① Toggle - FF

T (Toggle)

Wenn $T=1$, kippe (toggle) den aktuell eingespeicherten Wert (aus 1 wird 0, aus 0 wird 1), bei $T=0$ halte den aktuellen Wert

E	Q_{t-1}	Q_t	
0	0	0	
0	1	1	
1	0	1	$0 \rightarrow 1$
1	1	0	$1 \rightarrow 0$

$$Q_t = E \oplus Q_{t-1}$$

RS (Reset, Set)

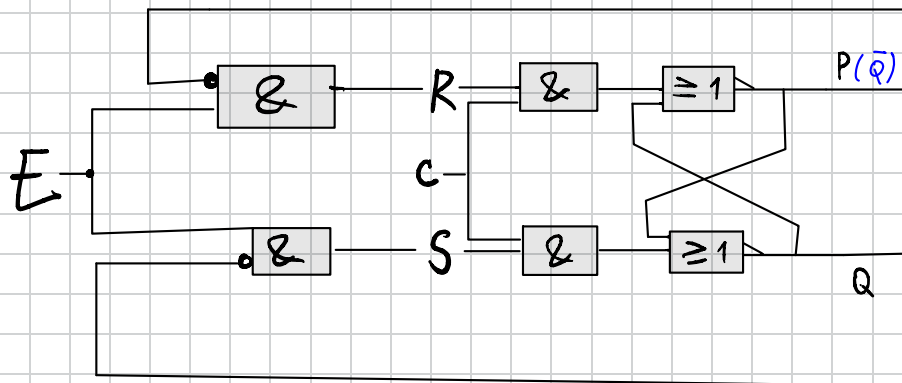
1: verboten

Wenn $S=1$: "1 einspeichern", wenn $R=1$: "0 einspeichern", beide 0: "halten", beide

E	Q_{t-1}	Q_t	S	R	
0	0	0	0	0	"halten"
0	1	1	0	0	"halten"
1	0	1	1	0	$0 \rightarrow 1$
1	1	0	0	1	$1 \rightarrow 0$

$$S = E \wedge \overline{Q_{t-1}}$$

$$R = E \wedge Q_{t-1}$$



Toggle-FF

1.3 Welche Eigenschaften sollte ein geeigneter Eingabeflipflop haben? Entspricht der Toggle-Flipflop diesen? Wenn nicht, welchen Flipflop empfehlen Sie?

Ein geeigneter Eingabeflipflop (als Teil eines Eingaberegisters) muss die Eigenschaft besitzen, den anliegenden externen Signalwert zuverlässig zu speichern und diesen Wert an das System weiterzuleiten, unabhängig von seinem vorherigen Zustand. Das Ziel ist die Übernahme des aktuellen Werts ins synchrone System.

Der Toggle-Flipflop (T-FF) ist per dieser Definition nicht geeignet, da er lediglich eine Aktion (Zustandswechsel) ausführt, wenn $E = 1$ anliegt. Wenn das externe Signal $E = 1$ ist und der T-FF bereits $Q_{t-1} = 1$ gespeichert hat, wechselt der Zustand zu $Q_t = 0$. Die gespeicherte Information wäre somit falsch.

Wir benötigen einen FF, der den Wert des Eingangs exakt übernimmt.

Empfehlung: Der D-Flipflop (Data Flipflop). Seine Funktion ist $Q_t = D$. Das bedeutet, der Ausgang Q speichert zuverlässig den an seinem Eingang D anliegenden Wert, unabhängig vom alten Zustand.

Aufgabe 2 *Automaten*

② Ampel

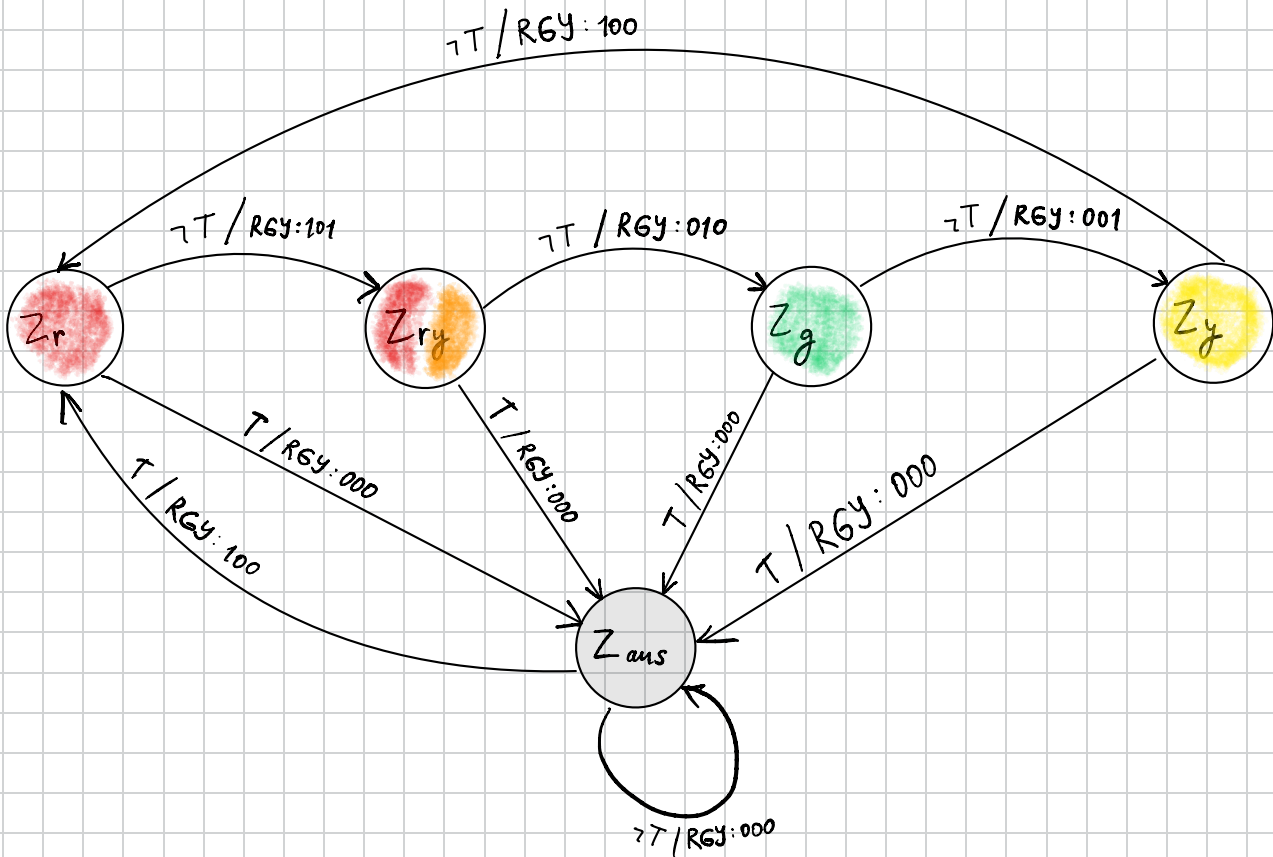
1) $Q = \{Z_r, Z_{ry}, Z_g, Z_y, Z_{aus}\}$

$E = \{T, \neg T\}$ (Taste gedrückt oder nicht)

$A = \{(1, 0, 0), (1, 0, 1), (0, 1, 0), (0, 0, 1), (0, 0, 0)\}$

	R	G	Y
rot	1	0	0
rot-gelb	1	0	1
grün	0	1	0
gelb	0	0	1
aus	0	0	0

2) Automat



3) Zustandsübergangsfunktion

	R	G	Y	T	R'	G'	Y'	
1	0	0	0	0	0	0	0	Bleibt aus
2	0	0	0	1	1	0	0	Rot
3	1	0	0	0	1	0	1	Rot-gelb
4	1	0	0	1	0	0	0	aus
5	1	0	1	0	0	1	0	grün
6	1	0	1	1	0	0	0	aus
7	0	1	0	0	0	0	1	gelb
8	0	1	0	1	0	0	0	aus
9	0	0	1	0	1	0	0	rot
10	0	0	1	1	0	0	0	aus
11	0	1	1	0	*	*	*	Nicht erreichbare Zustände markieren als „don't care“.
12	0	1	1	1	*	*	*	
13	1	1	0	0	*	*	*	
14	1	1	0	1	*	*	*	
15	1	1	1	0	*	*	*	
16	1	1	1	1	*	*	*	

4) Schaltnetz

K-Map R'

	0	0	1	1
00	0	1	0	1
01	0	0	*	*
11	*	*	*	*
10	1	0	0	0

K-Map G'

	0	0	1	1
00	0	0	0	0
01	0	0	*	*
11	*	*	*	*
10	0	0	0	1

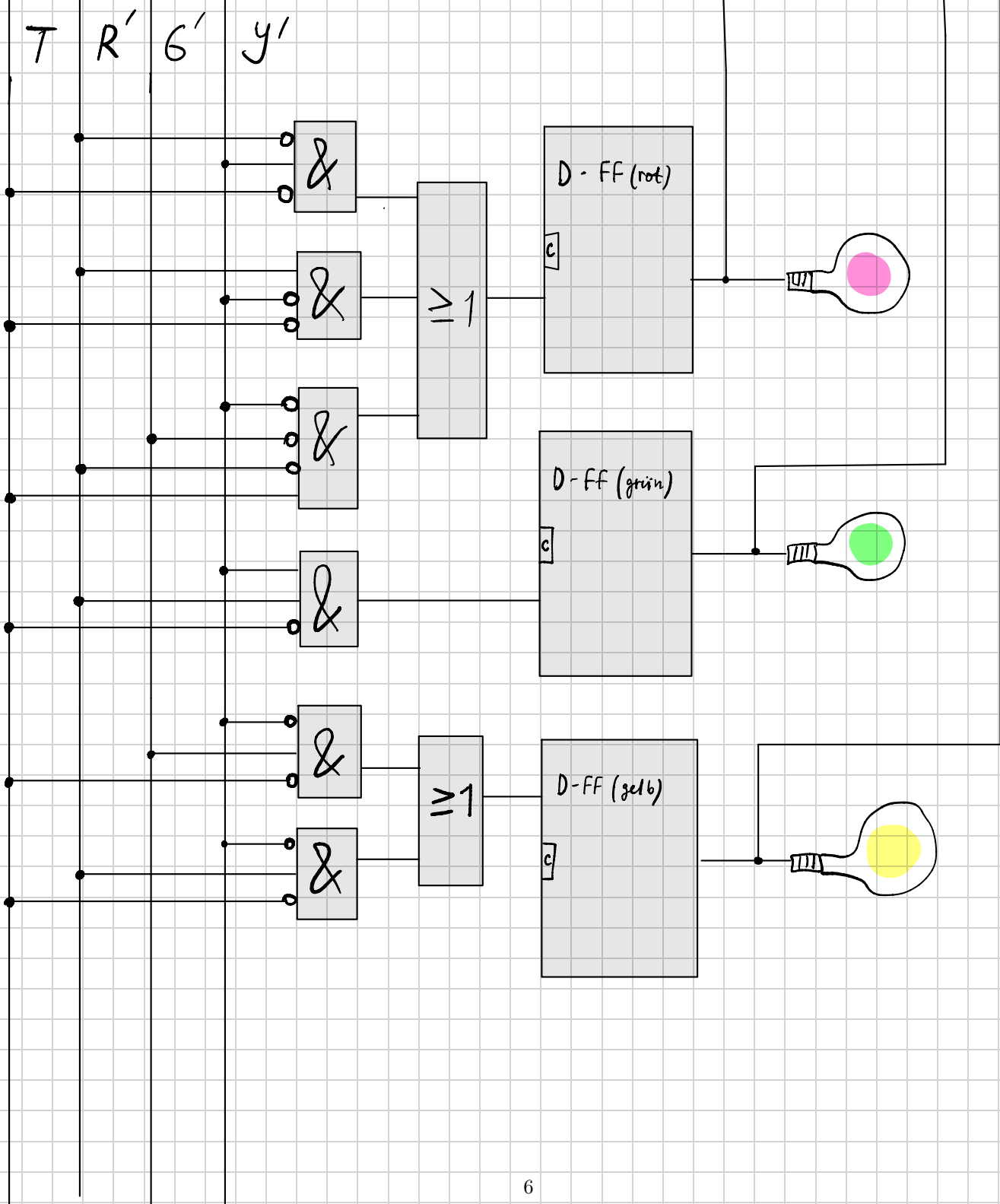
K-Map Y'

	0	0	1	1
00	0	0	0	0
01	1	0	*	*
11	*	*	*	*
10	1	0	0	0

$$R' = (\bar{R} \wedge Y \wedge \bar{T}) \vee (R \wedge \bar{Y} \wedge \bar{T}) \vee (\bar{R} \wedge \bar{G} \wedge Y \wedge T)$$

$$G' = (R \wedge Y \wedge \bar{T})$$

$$Y' = (G \wedge \bar{Y} \wedge \bar{T}) \vee (R \wedge \bar{Y} \wedge \bar{T})$$



2.5 Anpassung an Phasenlängen

Um verschiedene Phasenlängen zu ermöglichen, schaltet der Automat nicht nach jedem Takt weiter. Stattdessen nutzen wir ein **internes Zähler-Register** (Counter). Dieses Zähler-Register wird beim Eintritt in eine Phase mit der korrekten Zeit geladen. Der Zustandsübergang zur nächsten Phase findet dann nur statt, wenn der Zähler Null erreicht hat.

Bei sehr langen Phasen ist ein **externer Timer** besser. Dieser Timer sendet ein **Unterbrechungssignal** (Interrupt) an den Rechner, wenn die Zeit abgelaufen ist. Das spart CPU-Takte und macht die gesamte Steuerung effizienter.

Aufgabe 3 *Register-Transfer-Ebene*

3.1 Takt-Flanken-Effekt

Die getrennte Taktung erhöht die Zuverlässigkeit des Systems. Die ALU beginnt die Berechnung an der Vorderflanke des Taktsignals. Mit der Annahme Laufzeit = 0, die wir in diesem Modell benutzen, ist das Ergebnis der Berechnung dann auf der Hinterflanke verfügbar. Der neue Wert kann von den Registern gespeichert werden.

Stellt man sich diese Architektur so vor, dass die Register wie die ALU an der Vorderflanke getaktet werden, sieht man, dass es schwierig wird, den Zustand nach der Berechnung vorherzusagen. Es kann zu Race Conditions kommen.

3.2 Mikroinstruktionen ABC

Aufgabe 3

Aufgabe 4

Aufgabe 5

Mikroinstruktion	A	B	C	D	E	F	G	H	I	B
ALUINP-1-RAM	0	0	0	0	0	0	0	1	0	0
ALUINP-1-A	0	0	1	0	0	0	0	0	1	0
ALUINP-1-D1	0	0	0	0	0	0	1	0	0	0
ALUINP-1-D2	0	0	0	0	0	1	0	0	0	0
ALUINP-1-D3	0	0	0	0	1	0	0	0	0	0
ALUINP-1-0	1	1	0	1	0	0	0	0	0	1
ALUINP-1-1	0	0	0	0	0	0	0	0	0	0
ALUINP-2-D1	0	0	0	1	0	0	0	0	0	0
ALUINP-2-D2	0	1	0	0	0	0	1	0	0	1
ALUINP-2-D3	0	0	0	0	0	0	0	0	0	0
ALUINP-2-0	1	0	0	0	1	0	0	1	0	0
ALUINP-2-1	0	0	1	0	0	1	0	0	1	0
ALU-ADD	1	1	1	1	1	1	1	1	0	1
ALU-SUB	0	0	0	0	0	0	0	0	1	0
RAM-W	0	1	0	0	0	0	0	0	0	1
A-W	0	0	1	0	1	0	0	0	1	0
D1-W	0	0	0	0	0	0	0	0	0	0
D2-W	0	0	0	1	0	1	0	1	0	0
D3-W	1	0	0	0	0	0	1	0	0	0

Tabelle 1: Mikroinstruktionen

3.3 Mikroprogramm AEDBCFB

3.3. a) A - E - D - B - C - F - B

A :	0 → D3	D3 = 0
E :	D3 → A	A = 0
D :	D1 → D2	D2 = 4711
B :	D2 → RAM	RAM 0 = 4711
C :	A + 1	A = 1
F :	D2 + 1	D2 = 4712
B :	D2 → RAM	RAM 1 = 4712

RAM 0	4711
RAM 1	4712
RAM 2	68
...	
D1	4711
D2	4712
D3	0
A	1

3.3. b) ~~A~~ - ~~E~~ - D - B - C - F - B

A und E - redundante

3.3. c)

Wir gehen davon aus, dass die Mikroinstruktionen frei von Nebenwirkungen (side-effects) sind, d.h. keine weiteren Effekte auf das System haben, als die in der Aufstellung angegebenen.

Entsprechend können wir eine Mikroinstruktion ausgelassen, wenn sie eine No-Op darstellt. Beispiel: in einem Ausgangszustand von $D_2 = D_1$ kopieren wir den Wert in D_1 nach D_2 . Der Zustand des Systems bleibt gleich. Genau das passiert bei den ersten beiden Mikroinstruktionen A und E .

3.4. $D_3 = D_1 + D_2$

Siehe oben: Tabelle 1, Mikroinstruktion G .

3.5. $A[i - 1] = A[i]$

Siehe oben: Tabelle 1. Die Instruktion besteht aus der Folge von Mikroinstruktionen: H, I, B .

⑤ Ziel:

$RAM[A] \rightarrow D_2 \dots D_2 \rightarrow RAM[A-1]$

H.

ALU1 - RAM

ALU2 - 0

ALU - ADD

D2 - W

I $A = A - 1$

ALU1 - A

ALU2 - 1

ALU - SUB

A - W

B $D_2 \rightarrow RAM$

3.5. a)

Bei $A = 0$ resultiert die Operation $A - 1$ in einem negativen Ergebnis. Das System könnte auf verschiedene Weise reagieren:

- Wrap-around: Durch Modulo-Arithmetik das Ergebnis als höchsten Wert interpretieren (bei 4-bit 1111).
- Sättigung: Das Ergebnis auf den niedrigsten Wert festsetzen (d.h. A bleibt in diesem Fall 0.)
- Ausnahmebehandlung: Das Ergebnis als unzulässig betrachten und entsprechend vorgehen: z.B. Programmausführung stoppen, Fehler signalisieren.

Jede dieser Strategien bringt Komplikationen mit sich, z.B. kann bei einem wrap-around die neue Adresse undefinierte Daten enthalten oder liegt möglicherweise außerhalb des erlaubten Adressbereichs

3.5. b)

In dieser Architektur spielt eine große Rolle, dass die Konstanten 0 und 1 in der ALU verfügbar sind: wir können mit wenig Aufwand kopieren und de-/inkrementieren.

Der Ablauf wird auch dadurch erleichtert, dass wir die Komponenten ohne ein Kontrollsignal lesen können.

3.5. c)

Die Instruktionen bestehen aus unterschiedlich vielen Mikroinstruktionen. In unserem Modell braucht eine Mikroinstruktion stets eine feste Zeit (s. Folie 84 in Kapitel 6 - Register-Transfer-Ebene). Daraus folgt, dass die beiden Instruktionen unterschiedlich lange brauchen.